

# NEUROQUEST

## The 3D Spatial Memory Palace for Education

College project report

For students, lecturers, and project evaluators



Concept illustration of a memory palace. The image is not a screenshot of the current application.

### Main purpose

Help students organize academic concepts in memorable locations and practise retrieving them before checking their notes.

**Prepared for:** a college project presentation

**Presented by:** NeuroQuest Project Team

**Implementation reviewed:** 19 September 2026

This report describes the implemented website. It labels proposed backend services and future educational evaluation separately.

# 1. Project overview and purpose

NeuroQuest is a browser-based study application that connects notes with objects inside a navigable 3D space.

## Abstract

A memory palace associates information with places. NeuroQuest brings this idea into an interactive website: students create subject rooms, place memory anchors inside them, attach study notes, and return to those locations during revision. Explore mode supports reading and navigation. Recall Challenge hides titles and asks students to identify the concept before revealing the note. The student then records a self-assessment.

The project combines Next.js, React, TypeScript, and Three.js with a shared Zustand store. It saves rooms, anchors, and selected progress data in browser localStorage. The current implementation does not include an application backend, authentication, or a cloud database. It supports a usable HTML study interface when WebGL is unavailable.

## Why build this website?

The central purpose is to make deliberate recall easier to practise. A student needs more than a place to collect notes: a revision tool should also prompt them to check what they can explain without looking. NeuroQuest supplies spatial cues and a repeatable study sequence. Its 3D setting gives each idea a visible location, while the recall workflow creates a pause before the answer appears.

## Project objectives

### 1. Organize knowledge

Provide separate palaces for different subjects and focused anchors for individual concepts.

### 2. Support recall practice

Hide concept labels until the student chooses to reveal them, then collect a self-rating.

### 3. Make the system usable

Provide responsive forms, keyboard-accessible controls, readable notes, and a graphics fallback.

### 4. Demonstrate sound engineering

Separate UI, 3D graphics, state, validation, and persistence into maintainable modules.

**Scope:** an individual study and revision tool. It supplements teaching and problem solving. It is not a replacement for a learning management system, a teacher, or formal assessment.

## 2. Learning idea and intended users

The design combines spatial associations with a prompt to retrieve information before revealing it.

### Method of Loci

The Method of Loci associates information with distinct locations along an imagined or familiar route. In this project, the student uses a digital room instead: a selected artifact and its position become cues for an academic idea. The title and full explanation remain attached to the anchor. The student must build the association through study; simply placing an object does not create understanding.

### Active recall

Recall Challenge asks the student to attempt an answer before opening the note. Roediger and Karpicke (2006) found benefits of retrieval practice on delayed tests in experiments using prose material [R1]. Dresler and colleagues (2017) studied mnemonic training using the Method of Loci and reported memory improvements in that research setting [R2]. These studies motivate the design. Neither study evaluated NeuroQuest, and the project has no measured claim about examination marks or memory improvement.

### Who can use it?

| Audience             | Possible use                                                                                             |
|----------------------|----------------------------------------------------------------------------------------------------------|
| College students     | Organize definitions, terminology, formulas with conditions, programming concepts, or historical events. |
| Lecturers            | Demonstrate a revision method and ask learners to explain the concept behind an anchor.                  |
| Project evaluators   | Examine interaction design, state management, data validation, and browser graphics.                     |
| Independent learners | Create personal subject rooms and practise at a chosen pace.                                             |

### Good learning practice

Keep each anchor focused. Read for understanding, explain the concept in your own words, and solve relevant exercises. During recall, answer aloud or on paper before revealing the note. Revisit difficult concepts in later sessions. The current app does not calculate a revision schedule or verify that an explanation is correct.

Research references R1 and R2 appear in the final section. All proposed educational uses require suitable content and honest self-assessment.

### 3. Functional features and scope

A subject palace contains memory anchors. Each anchor combines a study note with a position and a visual artifact.

| Feature                | Current behavior                                                                       |
|------------------------|----------------------------------------------------------------------------------------|
| Subject selection      | Three example palaces: Computer Science, Human Anatomy, and World History.             |
| Custom palaces         | Create a room with a unique name, optional description, icon, and color.               |
| Anchor creation        | Add a title, category, Markdown material, shape, color, and coordinates.               |
| Explore mode           | Navigate the room, focus an anchor, and read or edit its note.                         |
| Recall Challenge       | Hide titles, reveal a concept after an attempt, and record Easy, Hard, or Failed.      |
| Search and map         | Find notes across palaces in Explore mode or focus an anchor using the spatial map.    |
| Progress and settings  | View room totals, self-rated mastery, effects settings, and an optional ambient sound. |
| Persistence and export | Keep selected data in the same browser and download a JSON snapshot.                   |

#### Content and limits

The six artifact shapes are crystal, ring/torus, cube, sphere, pyramid, and knot. Color choices and coordinates help distinguish locations. The store permits up to 50 palaces and 300 anchors across the workspace. Palace names can contain up to 48 characters. X and Z positions must stay between -6 and 6, and Y between 0 and 3.

The default subjects contain demonstration study notes. Human Anatomy uses geometric cues rather than a detailed anatomical model. World History uses notes in the shared vault environment rather than historically reconstructed locations. New subject palaces use the same scene system with their own content.

**Not included:** accounts, automatic grading, AI-generated answers, collaborative editing, full historical review logs, scheduled spaced repetition, or an import screen. These should not be described as completed features.

# 4. Student user guide

A useful first session starts with one subject and a small number of clear concepts.

## 1. Open NeuroQuest

Open the running website in a modern browser. Select a subject in the sidebar. On a phone, open the navigation menu first.

## 2. Create a subject if needed

Select **Create palace**. Enter a unique name, an optional description, an icon, and a color. Submit the form. The new empty palace opens immediately.

## 3. Add one memory anchor

Select **Add memory anchor** or **Place a new memory**. Enter a title and study material. Add an optional category. Choose a shape and color. Use the suggested position or adjust valid coordinates, then select **Add to palace**.

## 4. Learn in Explore mode

Drag to orbit and scroll to zoom. On touch devices, use one finger to orbit and pinch to zoom. Select an artifact, its label, a map point, or its list entry to read the note. Use Reset camera to return to the room view.

## 5. Attempt Recall Challenge

Switch to Recall Challenge. The room becomes darker and titles are hidden. Select an anchor and answer the prompt before selecting **Reveal concept**. Search results are unavailable during the challenge.

## 6. Rate the attempt

Select **Easy**, **Hard**, or **Failed**. Continue with **Next anchor**. The app allows one rating per anchor in the current session.

## 7. Review and return

Check Learning insights and revisit difficult material. Refreshing preserves saved notes and progress in the same browser, but resets open panels and the current recall-session details.

## Useful controls

| Action         | Control                                                  |
|----------------|----------------------------------------------------------|
| Search         | Search button or Ctrl+K / Cmd+K with study panels closed |
| Close a panel  | Escape or its close button                               |
| Edit or delete | Open the anchor drawer in Explore mode                   |
| Export data    | Workspace settings, then Export all palaces              |

# 5. Worked example and classroom use

A spatial cue helps the student return to an idea. Explanation and application complete the learning activity.

## Example: Binary Trees

### 1. Choose a memorable cue

Use the Computer Science palace and associate Binary Trees with a distinctive artifact near a chosen corner.

### 2. Write a focused note

State that each node has at most two children. Add a small example and explain root, leaf, and subtree. A fenced code block can hold pseudocode or a short implementation.

### 3. Build the association

Read the note, draw the structure, and explain the definition without copying it. Notice the artifact and where it sits in the room.

### 4. Recall before reveal

In Recall Challenge, look at the artifact and identify the concept. Explain the rule about children before opening the note. Compare the answer and give an honest rating.

### 5. Apply the concept

Solve a traversal problem outside the app or include an exercise in the note. Remembering a title alone does not demonstrate full understanding.

## Other subject examples

| Subject       | Possible anchor content                                                          |
|---------------|----------------------------------------------------------------------------------|
| Human Anatomy | Names and functions of a selected body system, reviewed against course material. |
| World History | An event with its date, causes, consequences, and supporting course references.  |
| Mathematics   | A theorem, the conditions where it applies, and a worked example.                |
| Languages     | A word or phrase with meaning, pronunciation guidance, and an example sentence.  |

## A lecturer-led activity

Introduce a short topic, then ask students to choose five key concepts and create one anchor for each. Allow time to explain the associations. In a later session, ask students to recall and apply the ideas before viewing their notes. Discuss mistakes in class. This is a proposed teaching activity, not an implemented teacher dashboard or a validated study result.

# 6. Programming languages and stack

TypeScript is the main programming language. Frameworks, runtimes, and data formats have separate roles.

| Technology                  | Type and purpose                                                                             |
|-----------------------------|----------------------------------------------------------------------------------------------|
| TypeScript / TSX            | Typed application logic and React components. TSX combines TypeScript with component markup. |
| JavaScript                  | The browser executes JavaScript produced from the TypeScript source.                         |
| HTML / CSS                  | Document structure and visual styling. CSS handles responsive layout and transitions.        |
| Next.js 16 / React 19       | Application framework, App Router entry, and interactive component system.                   |
| Three.js / R3F 9            | Three.js provides the 3D renderer. React Three Fiber manages its scene through React.        |
| Drei / postprocessing       | Orbit controls, scene helpers, Bloom, Vignette, and ChromaticAberration.                     |
| GSAP                        | Camera-position and orbit-target transitions.                                                |
| Zustand 5                   | Shared store with actions, selective subscriptions, and persistence.                         |
| Tailwind CSS 4 / Lucide     | Styling support and UI icons alongside custom CSS.                                           |
| React Markdown / remark-gfm | Render Markdown notes, including lists, tables, and fenced code blocks.                      |
| Node.js / npm               | Run development tools, install packages, build, and serve the application.                   |
| JSON                        | A data format for browser persistence, settings, and export.                                 |

Development checks use TypeScript, ESLint, Vitest, and Prettier. DM Sans and Space Grotesk fonts are bundled locally. Exact package declarations appear in package.json and resolved versions in package-lock.json.

**Backend language:** none is implemented for application data. Node.js runs the Next.js tooling/server, but that alone does not create an authenticated study API or a database.

# 7. Frontend architecture and modules

The interface, 3D scene, and state store have separate responsibilities.

| File or module                    | Responsibility                                                                      |
|-----------------------------------|-------------------------------------------------------------------------------------|
| src/app/page.tsx                  | App Router page that mounts the workspace.                                          |
| src/app/layout.tsx                | Shared document layout, metadata, local fonts, and global styles.                   |
| NeuroQuest.tsx                    | Workspace composition, map, search, settings, learning insights, and scene loading. |
| Navigation / ModeSwitcher         | Subject navigation, new-palace entry point, and study-mode controls.                |
| AddRoomModal / AddAnchorModal     | Forms for creating subjects and memory anchors.                                     |
| AnchorDrawer / Dialog             | Notes, edits, recall prompts, ratings, focus containment, and panel transitions.    |
| Scene / MemoryAnchor              | 3D environment, lighting, objects, animation, and the graphics fallback.            |
| CameraController / PostProcessing | Camera transitions, navigation bounds, and optional visual effects.                 |
| usePalaceStore.ts                 | State, actions, validation, persistence, and data migration.                        |
| data.ts / types.ts                | Example content and TypeScript domain definitions.                                  |

## Interaction sequence

When a student selects an anchor, the UI calls a store action. The store updates the selected anchor and camera target. The drawer reads the selected note, while the camera controller reads the target and moves the view. Both respond to the same source of state without placing camera animation frames inside React state.

## Rendering boundaries

NeuroQuest dynamically imports Scene with server rendering disabled. WebGL therefore initializes in the browser. Forms keep unfinished input in component state. Components subscribe to individual Zustand values or actions, which avoids publishing every form keystroke to the scene. The shared store still updates subscribers when relevant data changes.

## 8. 3D rendering and interaction

The environment gives notes stable visual locations while keeping study controls available outside the canvas.

### Scene and anchors

Scene creates procedural architecture, a ground grid, pedestals, and atmospheric lighting. A subdued ambient base combines with directional and localized accent lights. Recall mode reduces ambient, hemisphere, and directional intensity. Each MemoryAnchor renders one of six geometric artifact types. Idle movement uses subtle vertical hovering and slow rotation through the render loop. Hovering changes visual emphasis and displays a label.

### Camera behavior

OrbitControls applies damping, bounded zoom, and polar-angle limits of 24 to 78 degrees. Panning is disabled to keep the view within the room. Selecting an anchor sets a target for a 1.35-second GSAP transition using power3.out. The controller cancels a tween when the user starts orbiting, the target changes, a reset occurs, or the component unmounts. Initial framing and reduced-motion behavior apply immediately.

### Effects and performance choices

PostProcessing combines Bloom, Vignette, and subtle ChromaticAberration. Bloom uses luminanceThreshold 0.6 and intensity 1.2. Students can disable effects in settings. Rendering limits pixel ratio to 1.5 for fine-pointer devices and 1.25 for coarse-pointer devices, uses 1024-pixel shadow maps, and disables multisampling in the effect composer. These are engineering choices, not measured frame-rate guarantees.

### Resource management

React Three Fiber manages disposal of declarative geometry and material resources when they unmount. Camera tweens stop when no longer needed. Locally generated ambient audio closes its resources when disabled or unmounted. Long sessions and repeated navigation should still receive device-specific graphics testing.

### Accessibility and fallback

The HTML interface provides anchor lists, a spatial map, keyboard-accessible controls, and a shared dialog with focus containment and restoration. Reduced-motion preferences stop idle animation and shorten UI motion. If WebGL fails or its context is lost, an HTML anchor view preserves study operations and offers Retry 3D view. Decorative HUD containers allow pointer events through to the scene, while buttons receive input.

## 9. Data model and persistence

A room owns anchors through `roomId`. Retention and review information belong to each anchor.

| Entity                  | Important fields                                                          |
|-------------------------|---------------------------------------------------------------------------|
| Room                    | id, name, subtitle, color, icon                                           |
| MemoryAnchor identity   | id, roomId, title, category, content                                      |
| MemoryAnchor appearance | shape, color, position [x, y, z]                                          |
| MemoryAnchor progress   | status, reviewCount, lastReviewedAt                                       |
| Study state             | active room, mode, selected anchor, camera target, recall-session ratings |
| Preferences             | Ambient sound and postprocessing effects                                  |

### What gets saved?

The persistence layer stores rooms, anchors, retention status, review counts, latest review timestamps, active room, mode, and selected preferences. Its `localStorage` key is **neuroquest-palace**, with storage schema version **2**. Version 1 data migrates automatically so older notes and progress remain available after the custom-palace feature.

Open panels, selected anchors, camera position, and current-session recall ratings are transient. The application records the latest review time and count, not a complete history of every answer. Easy sets status to mastered. Hard and Failed set status to learning. Each accepted rating increments the count and updates the time.

### Validation and recovery

Store actions validate user input. Hydration validates saved rooms before validating anchors, so room references must be valid. Checks cover identifiers, duplicates, names, colors, icons, coordinates, and data shape. Invalid records are discarded. Intentional empty rooms remain empty. If browser storage is unavailable, the live in-memory session can continue, but durable saving is not guaranteed.

### Data boundaries

Saved data is specific to the browser profile and origin. Another browser, another device, or a different local port has separate storage. Clearing site data removes saved content. JSON export provides a snapshot of rooms, anchors, and review data, but no import interface exists yet. Browser storage is not encrypted user-account storage and does not provide access control on a shared device.

# 10. Backend: current and proposed

The current application is a browser-based implementation with local persistence. A server-side study backend is future work.

## What exists now

Next.js supplies the application framework, development server, build process, and production serving. The browser executes the study logic and stores data. The source has no implemented study API routes, authentication flow, account system, or server database. No API keys or database configuration are required to run this version. Calling it a complete cloud-based full-stack platform would misrepresent its current scope.

## A possible future backend

| Layer                  | Proposed responsibility                                                                      |
|------------------------|----------------------------------------------------------------------------------------------|
| Authentication         | Identify a user through a maintained authentication system and secure session handling.      |
| Next.js Route Handlers | Expose authenticated operations for creating, reading, updating, and deleting study records. |
| PostgreSQL database    | Store users, rooms, anchors, and review events with constraints and ownership relationships. |
| Authorization          | Check that a user owns a room or has explicit permission before returning or modifying it.   |
| Synchronization        | Track versions or update times, resolve conflicting edits, and recover from failed requests. |
| Operations             | Maintain backups, environment secrets, monitoring, rate limits, and secure deployment.       |

## Proposed relational model

**User** owns many **Rooms**. A Room contains many **Anchors**. An Anchor has many **ReviewEvents**. ReviewEvent records could store the rating, review time, and user, enabling a complete history. Classroom sharing would need additional role and membership tables. This model describes a design option and is not present in the current application.

# 11. Build process and local setup

A modern browser and Node.js 20.9 or newer are sufficient for the current version.

## Implementation stages

The source reflects these stages: define TypeScript room and anchor models; implement a validated Zustand store; create the responsive workspace and study forms; build procedural 3D components; connect selection to camera focus; implement recall and ratings; add local persistence and migration; verify behavior with automated and browser checks. Separate modules make it possible to extend one layer without rewriting the entire application.

## Run the project

### 1. Install prerequisites

Install Node.js 20.9 or newer with npm. Use a browser with WebGL and hardware acceleration for the 3D experience.

### 2. Open the project folder

Open a terminal in the directory containing package.json. Install the locked dependencies:

```
npm ci
```

### 3. Start development

Run the development server, then open <http://localhost:3000> in the browser:

```
npm run dev
```

### 4. Build and serve production

Run these commands in order. npm start serves an existing production build:

```
npm run build
npm start
```

If port 3000 is already occupied, use `npm run dev -- --port 3001` and open that port. No API keys, accounts, environment variables, or database setup are needed for this implementation.

## Deployment considerations

The project can run on a hosting environment that supports the chosen Next.js build and runtime. The current localhost address is a local preview, not a public deployment. Saved browser data does not automatically transfer to another hosted origin. There is no implemented service worker or offline-installable PWA guarantee. Before a college demo, start the server and test the actual browser and display equipment.

# 12. Verification and educational evaluation

Software correctness and learning effectiveness are separate questions.

## Automated and manual checks

| Check                 | What it covers                                                                                                                                  |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 19 Vitest store tests | Ratings, duplicate recall prevention, room changes, camera reset state, anchor changes, validation, persistence, migration, and malformed data. |
| TypeScript and ESLint | Static type and source-quality checks.                                                                                                          |
| Prettier              | Consistent source formatting.                                                                                                                   |
| Production build      | Compilation and generation of the deployable application.                                                                                       |
| Browser checks        | Custom palace creation, duplicate-name feedback, first-anchor creation, refresh persistence, and responsive forms.                              |

```
npm run format:check
npm run lint
npm run typecheck
npm test
npm run build
```

The latest feature checks completed 19 store tests, lint, and the production build successfully. The browser used for the latest interface checks showed the HTML fallback. That confirms the study forms and fallback workflow, but does not validate real-time 3D performance across all GPUs. State tests also do not replace accessibility testing with users.

## Suggested classroom evaluation - not conducted

Use the same course material and study time for a comparison group and a NeuroQuest group. Include a baseline check, an immediate test, and a delayed recall test. Score answers with a predefined rubric rather than app mastery labels. Collect task completion, usability feedback, and device performance. With appropriate consent and institutional review where required, this could show whether the workflow helps the intended learners.

Do not report invented improvements, success percentages, student satisfaction scores, or examination gains. The project currently demonstrates functionality. Its educational effectiveness remains to be evaluated.

# 13. Advantages, limitations, and roadmap

The strongest implemented advantage is a customizable workflow that combines study material with a recall prompt.

## Advantages

**Personal organization:** separate palaces and categories make subjects easier to navigate. **Recall routine:** hidden titles encourage an answer attempt before reveal. **Rich notes:** Markdown supports structured explanations and code snippets. **Low setup burden:** no account or database is required. **Multiple access paths:** the scene, list, map, and keyboard controls support different ways to navigate. **Modular code:** clear responsibilities make further development easier.

## Limitations and their implications

| Limitation                   | Implication                                                                      |
|------------------------------|----------------------------------------------------------------------------------|
| Local browser storage        | No automatic backup or synchronization. Clearing site data can lose work.        |
| Self-rated mastery           | A mastered label is not proof of a correct explanation or examination readiness. |
| Graphics requirements        | Device capabilities can affect rendering. The HTML fallback remains important.   |
| Preparation effort           | Students must write accurate notes and form useful associations.                 |
| Incomplete teaching platform | No accounts, class dashboards, content moderation, or collaborative workflows.   |
| No learning outcome study    | Expected benefits should remain hypotheses until evaluated.                      |

## Prioritized future work

### 1. Data recovery

Add a validated import/restore flow before expanding the amount of user content.

### 2. Learning support

Evaluate the workflow and consider scheduled revision with fuller review histories.

### 3. Backend services

Add authenticated synchronization, ownership checks, backups, and controlled sharing.

### 4. Teaching and accessibility

Explore lecturer templates, classroom workflows, and usability testing with diverse learners.

# 14. Presentation demo and viva preparation

Use the editable PowerPoint for a 12-15 minute talk, with a short demonstration and time for questions.

## Suggested team presentation flow

**Opening:** explain what NeuroQuest is and why the project exists. **Student experience:** show a subject, an anchor, and the Explore/Recall difference. **Technical explanation:** describe TypeScript, the frontend modules, 3D rendering, and local persistence. **Closing:** state the limitations and future backend plan. Assign sections according to your team size; speaker notes in the deck provide a starting script.

## Three-minute demonstration

Open Computer Science and select one anchor. Create a new palace such as Mathematics if the name is available. Add a concise note for the Pythagorean theorem, including its right-triangle condition. Select an artifact and position. Switch to Recall Challenge, explain the idea before revealing it, and rate the attempt. Refresh to show persistence. Use the anchor list if the 3D view is unavailable.

## Likely lecturer questions

| Question                         | Suggested answer                                                                               |
|----------------------------------|------------------------------------------------------------------------------------------------|
| What is the main purpose?        | To help students attach concepts to places and practise recall before checking notes.          |
| Why use 3D?                      | It provides spatial cues and a navigable setting. Its learning benefit still needs evaluation. |
| Which language is used?          | Mainly TypeScript/TSX, with HTML and CSS. The browser runs compiled JavaScript.                |
| What is the backend?             | No application backend exists yet. Next.js serves the app and localStorage holds study data.   |
| Is this AI or automatic grading? | No. Students write notes and rate their own recall.                                            |
| Can it work on a phone?          | The UI supports small screens and touch. Graphics performance depends on the device.           |
| How will you improve it?         | Prioritize restore/import, educational evaluation, then secure accounts and synchronization.   |

# 15. References and glossary

Implementation statements come from the project source and its documented behavior.

## Project sources

- P1.** README.md: setup, architecture, workflow, limitations, persistence, and verification.
- P2.** package.json and package-lock.json: technologies, scripts, and dependencies.
- P3.** src/lib/types.ts and src/lib/data.ts: domain models and example content.
- P4.** src/store/usePalaceStore.ts: state actions, constraints, saving, and migration.
- P5.** src/components/NeuroQuest.tsx and ui/: workspace, dialogs, notes, and custom rooms.
- P6.** src/components/three/: rendering, camera, artifacts, and effects.
- P7.** tests/store.test.ts: automated state tests.

## Learning research

- R1.** Roediger, H. L., III, & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3), 249-255.  
<https://doi.org/10.1111/j.1467-9280.2006.01693.x>
- R2.** Dresler, M., et al. (2017). Mnemonic training reshapes brain networks to support superior memory. *Neuron*, 93(5), 1227-1235.e6.  
<https://doi.org/10.1016/j.neuron.2017.02.003>

## Glossary

| Term                  | Meaning in this project                                                             |
|-----------------------|-------------------------------------------------------------------------------------|
| Memory anchor         | An object, its location, and its attached study note.                               |
| Frontend              | The interface and interactive behavior that run in the browser.                     |
| Backend               | Server-side APIs, authorization, and data services. Proposed, not implemented here. |
| WebGL                 | A browser graphics interface used by the 3D renderer.                               |
| State                 | Current application data, such as the selected room or mode.                        |
| Persistence           | Keeping selected data between page loads.                                           |
| Hydration / migration | Loading saved state and adapting older data to a newer storage schema.              |

Cover art: generated concept illustration created for this presentation with the built-in Image Generation tool. All other descriptions refer to the project or explicitly labelled proposals.